Tu app de gimnasio con IA, desde cero y gratis

Guía paso a paso: cómo construí —con un agente de IA— la app que registra mi entrenamiento y genera los partes semanales de mi entrenador personal





0 · Qué vas a construir

Una web que funciona como app en el móvil con la que:

Cada día	Registras tu peso y el entrenamiento del día: eliges la sesión (FB1/FB2/FB3), ves las **series y reps pedidas** por tu entrenador junto a cada ejercicio, y apuntas el peso y las reps que has hecho.
Cada semana	Rellenas medidas corporales y un cuestionario de 9 preguntas. Pulsas **Generar informe** y la app produce los 4 documentos que espera tu entrenador (2 Excel + 1 Excel + 1 Word), con su formato original, y prepara el mensaje de WhatsApp.

Siempre	Todo se guarda en tu navegador. Sin servidores, sin base de datos, sin cuotas. Publicada gratis en GitHub Pages.

Requisitos previos: un Mac (o PC con Linux/Windows), conexión a internet y ganas de 2–3 horas. **Coste:** 0 € en hosting y librerías. La única cuenta de pago posible es la del asistente de IA (ver paso 2).

1 · Prepara tu material

Reúne en una carpeta los documentos que hoy rellenas a mano para tu entrenador (en mi caso: control de peso semanal, seguimiento de medidas, plan de entrenamiento en Excel y cuestionario en Word). Estos archivos serán las **plantillas**: la app los leerá y los rellenará con tu formato exacto.

```
mkdir ~/orca/mi-gym && cd ~/orca/mi-gym
# copia aquí tus .xlsx y .docx tal cual te los da tu entrenador
```

2 · Instala las herramientas

El IDE: Visual Studio Code

Descarga **VS Code** (code.visualstudio.com) y arrástralo a Aplicaciones. Ábrelo y abre tu carpeta del proyecto: `Archivo → Abrir carpeta…`. Es donde verás el código que genera la IA y donde puedes retocarlo.

El motor: Node.js

Instala Node.js LTS (nodejs.org). Comprueba en un terminal:

```
node --version   # debería responder v22 o superior
```

El asistente: Claude Code

Claude Code es un agente de programación que trabaja dentro de tu proyecto desde el terminal: lee tus archivos, escribe el código, ejecuta tests y te va contando lo que hace. Instalación:

```
npm install -g @anthropic-ai/claude-code
cd ~/orca/mi-gym
claude
```

La primera vez te pedirá iniciar sesión con tu cuenta de Claude (hay plan gratuito con límites; los planes de pago quitan el hartazo de límites). A partir de aquí, todo lo que sigue son **prompts que le escribes en lenguaje normal** — los tienes copy-paste en esta guía.

3 · Sesión 1: analiza tus plantillas y crea el prototipo

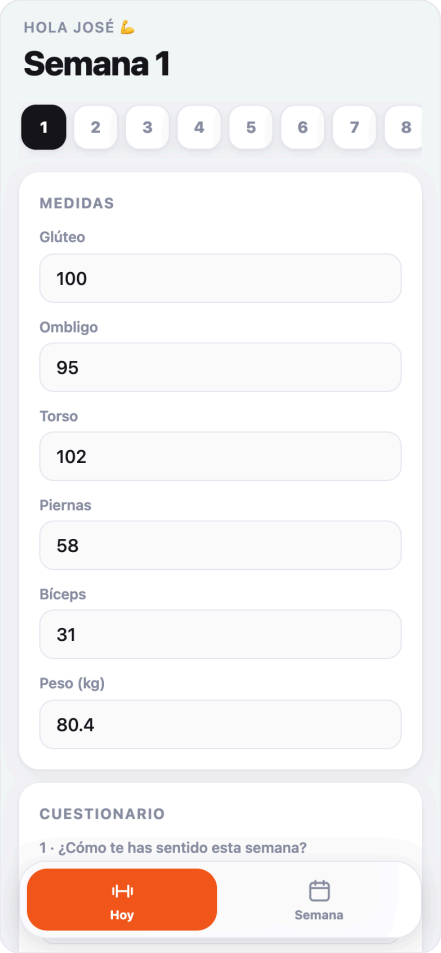
Con Claude Code abierto en tu carpeta, pega este prompt (adapta los nombres de tus archivos):

```
Tengo estos documentos que relleno cada semana para mi entrenador personal:
[control_de_peso.xlsx, medidas.xlsx, entrenamiento.xlsx, cuestionario.docx].
Es un rollo ir entre Drive, el ordenador y WhatsApp.

Quiero una app donde registre mi peso diario y mi entrenamiento,
y que al final de la semana genere esos 4 documentos rellenados
para enviarlos por WhatsApp a mi entrenador.

Empieza analizando la estructura de las plantillas y propón el plan.
```

El agente inspeccionará las celdas exactas de cada Excel, la estructura del Word, y te propondrá la arquitectura. En mi caso eligió: aplicación Python con Flask que guarda los datos en un JSON y rellena copias de las plantillas con openpyxl y python-docx. Deja que trabaje y luego **prueba tú**: abre la URL que te indique (algo como `http://localhost:5000`) en el navegador.



Qué comprobar en esta fase: que puedas registrar el peso del día, rellenar medidas y cuestionario, y que el botón de generar informe produzca 4 archivos correctos. Si algo no cuadra con tus plantillas, díselo: *"el peso de la semana debe ir en la fila X, columna Y"*. La IA corrige sobre la marcha.

4 · Sesión 2: diseño de app de verdad y registro por días

El primer prototipo funciona pero es feo y no se ajusta a cómo entrenas. Segunda tanda de prompts:

```
Ahora vamos a cuidar el diseño, optimizado para móvil, como si fuera
una app que vamos a vender. Revisa los estándares de apps de fitness
del mercado (Hevy, Strong): barra de pestañas inferior, controles
segmentados, tipografía 16px+ para evitar el zoom de iOS, safe-areas.

Además el entrenamiento debe ser editable por día: no hago todos los
ejercicios el mismo día. Lee los días (Día 1, Día 2, Día 3) del propio
Excel del plan y muestra solo los ejercicios de cada día, con las
series y reps pedidas al lado, y campos para el peso y las reps que
he hecho en cada serie.
```



Resultado: cabecera estilo iOS, selector segmentado FB1/FB2/FB3, tarjetas por ejercicio con las series pedidas (chips con series, reps, descanso y RIR) y campos de registro grandes para el pulgar. El plan se **lee directamente del Excel del entrenador**, así que cuando cambie la rutina, la app se actualiza sola.

> **Consejo:** itera en frío. Mira la app en el móvil y pide cambios concretos: "el morado no, usa un naranja tipo gym", "los campos de reps deben aceptar varias series: 10, 9, 8". Cada iteración tarda menos que decidir qué quieres cenar.

---

5 · Sesión 3: fichas de ejercicios con GIF (bonus)

```
Integra un dataset abierto de ejercicios para que cada ejercicio
tenga una ficha desplegable "¿Cómo se hace?" con GIF animado e
```

```
instrucciones en español. Usa el dataset de github.com/hasaneyldrm/
exercises-dataset (1.324 ejercicios, MIT) y mapea los nombres en
español de mi plan a los del catálogo.
```



GIF animado del movimiento, músculos trabajados e instrucciones paso a paso en español (el dataset trae 10 idiomas). Solo hizo falta mapear los 19 nombres de mi rutina; el resto es automático.

---

6 · Sesión 4: la convertimos en web estática y la publicamos gratis

Una app Python necesita un servidor. Para que viva gratis para siempre, se convierte a **web estática**: los datos se guardan en el navegador (localStorage) y los documentos se generan en el propio navegador con las librerías JavaScript ExcelJS y docx. Prompt:

```
Convierte la app a una web 100% estática para publicarla en GitHub
Pages: mismos datos en localStorage con export/import de copia de
seguridad JSON, y generación de los 4 documentos en el navegador
con ExcelJS (xlsx) y docx (Word). Sustituye la lógica de Flask y
deja la app lista para subir.
```

Y para publicar (crea un repo público; tus *datos* nunca se suben, se quedan en tu navegador):

```
git init && git add -A && git commit -m "mi app de gym"
gh repo create TUUSUARIO/mi-gym --public --source=. --push
gh api repos/TUUSUARIO/mi-gym/pages -X POST \
  -f "source[branch]=main" -f "source[path]=/"
```

Un minuto después: `https://tuusuario.github.io/mi-gym/`. En el iPhone: Compartir → **Añadir a pantalla de inicio** y la tienes como app.





7 · Pruébalo todo antes de fiarte

✓	Registra un día de entrenamiento con 2-3 ejercicios y comprueba la tira semanal.
✓	Rellena medidas y cuestionario de la semana 1.
✓	Pulsa **Generar informe**: se descargan los 4 documentos. Ábrelos y compara celdas con las plantillas originales.
✓	Pulsa **Enviar por WhatsApp**: se abre el chat con el resumen. Adjunta los archivos.
✓	Exporta la copia de seguridad JSON y guárdala. Es tu "drive" ahora.

Pedirle al agente que escriba **tests automáticos** ("añade pruebas con Node de la generación de documentos") es lo que te ahorra las sorpresas de un domingo a las 22:47.

8 · Problemas típicos y salida

Síntoma	Solución
`git@github.com: Permission denied (publickey)`	Tu SSH no está configurado: usa el remoto HTTPS (`git remote set-url origin https://github.com/TUUSUARIO/mi-gym.git`), que autentica con el token de `gh` .
No cargo la web en el iPhone	Usa la URL de GitHub Pages (no localhost) o la IP local del ordenador en la misma red Wi-Fi.
Cambié de móvil y no están mis datos	Los datos viven por navegador: importa el JSON de copia de seguridad.
El entrenador cambió el plan de ejercicios	Actualiza el Excel plantilla y pídele a la IA que regenere los datos del plan; el resto es automático.

9 · Costes, tiempo y lo que aprendí

	Antes	Ahora
Tiempo semanal	~20 min entre Drive, Excel y Word	~1 min (registro diario + 1 botón)
Coste	0 € y frustración	0 € de hosting (Pages); la cuenta de IA es lo único de pago
Stack	—	HTML + CSS + JS, ExcelJS, docx, localStorage, GitHub Pages

La lección no es "sé programador". Es que **describir bien un problema aburrido se ha convertido en una habilidad productiva**: el agente escribió todo el código; yo aporté las plantillas y el criterio de diseño.

Importante: esto es un **MVP personal**, un proyecto de un fin de semana para un problema concreto (una persona, sus datos en su navegador). Cuando el problema es de verdad profesional —varios usuarios, base de datos, integraciones, facturación— el proyecto pide otro nivel: arquitectura, seguridad, mantenimiento. Para eso existe **lumicatech.es**, donde estaremos encantados de escuchar cualquier problema y buscar la solución adecuada.